Assignment 2 - Image Filtering and CNN

Danial Bahrami Siavashani

CSCI 611 – Applied Machine Learning

Instructor: Dr. Bo Shen

Submission Date: April 5, 2026

1. Introduction

This assignment had two main parts. In Part 1, I worked on image filtering and completed the TODOs in the provided notebook. In Part 2, I built and trained a CNN model for image classification. These two parts were different, but they connect well because both deal with understanding and processing image data.

While doing this assignment, I got to see both sides of the topic. The first part was more about directly changing and analyzing images with filters, and the second part was about training a neural network to learn image features automatically. In this report, I go over what I implemented, what results I got, and what I noticed from both parts.

2. Background

Image filtering and CNNs are related because both are used to work with images and extract useful information from them. In regular image filtering, we apply a filter directly to an image to make certain features stand out, like edges, smooth areas, or texture. In a CNN, the model also works with image features, but instead of using fixed filters by hand, it learns useful filters during training.

This assignment included both of these ideas. Part 1 helped me understand the basics of image processing more clearly, especially how filters affect the image output. Part 2 took that idea further by using a CNN to classify images. Doing both parts in the same assignment helped me see the connection between basic image operations and deep learning.

2.1 Part 1 Overview - image_filtering

The image filtering part of the assignment focused on applying filtering operations to input images and observing how the outputs changed. These filters help reveal certain features of the image, such as edges, texture, smoothing effects, or noise reduction. This part was useful for understanding how pixel-level operations can affect the appearance and structure of an image.

By completing this section, I was able to see how different filters produce different visual results and why filtering can be helpful in image analysis tasks. Even though these operations are simpler than a CNN, they provide a strong foundation for understanding how image features can be detected and enhanced.

2.2 Part 2 Overview - build_cnn

The CNN part of the assignment focused on building and training a neural network for image classification. A CNN is well suited for image data because it uses convolution layers to detect patterns such as edges, shapes, and more complex visual features. As the data moves through the network, the model learns increasingly useful representations of the input image for classification.

In this part, the model architecture, training process, and testing results were analyzed. Important results included training behavior, validation performance, accuracy, training time, and the total number of trainable parameters. This part showed how a CNN can automatically learn features from data and use them to classify images more effectively than manual filtering alone.

3. Implementation Details

For this assignment, I completed the TODOs in both provided notebooks and ran each notebook fully to make sure the outputs, graphs, and tables were shown correctly. The two parts were different in purpose, but both were based on image-related tasks. Part 1 focused more on direct image processing, while Part 2 focused on training and evaluating a CNN model for classification.

3.1 Part 1 - image_filtering

In Part 1, I worked through the image_filter notebook and completed the missing TODO sections. This part mainly focused on applying filtering operations to images and observing how the output changed. The notebook used sample images, and the results could be seen directly after running the code. By comparing the original image and the filtered image outputs, it was easier to understand what each filter was doing.

This part was more visual than mathematical. The main idea was to see how image filters can affect details such as edges, smooth areas, and overall appearance. Some filters made the image look sharper, while others made it smoother or emphasized certain patterns. Completing this part helped me understand the basic idea of image feature extraction before moving to the CNN section.

3.2 Part 2 - build_cnn

In Part 2, I completed the build_cnn notebook and trained a convolutional neural network for image classification. The model was built step by step using convolution layers, activation functions, pooling layers, a flatten layer, and fully connected layers at the end for classification. As the input moved through the network, the spatial size became smaller while the number of feature maps increased, which allowed the model to learn more useful image features.

The CNN took a 32 x 32 x 3 input image and processed it through three convolution blocks with max pooling. After the final pooling layer, the output became 4 x 4 x 128, which was flattened into 2048 features before being passed into the fully connected layers. The total number of trainable parameters in the model was 1,147,466.

For training, I used CrossEntropyLoss as the loss function because this was a classification task. The optimizer setup followed the notebook example, and the assignment also suggested comparing with Adam. During training, the notebook displayed the training loss and validation loss for each epoch, and the model saved the best version whenever the validation loss improved. After training, the model was tested and the notebook showed the final results, including accuracy, graphs, and model summary information. This part helped me understand not only how to build a CNN, but also how to evaluate whether the model was learning effectively.

4. Results and Findings

4.1 Part 1 Results

For Part 1, the main result was being able to complete the image filtering TODOs and run the notebook successfully from start to finish. The outputs clearly showed how the filtered images changed compared to the original images. This part was more visual than numerical, so the results were easier to understand by directly looking at the images instead of focusing on metrics.

From this part, I could see that different filters changed the image in different ways. Some outputs made edges or details stand out more clearly, while others made the image smoother. Overall, Part 1 helped me understand how basic image processing works and how small pixel-level changes can affect the final appearance of an image.

Figure 1. Image filtering results from Part 1

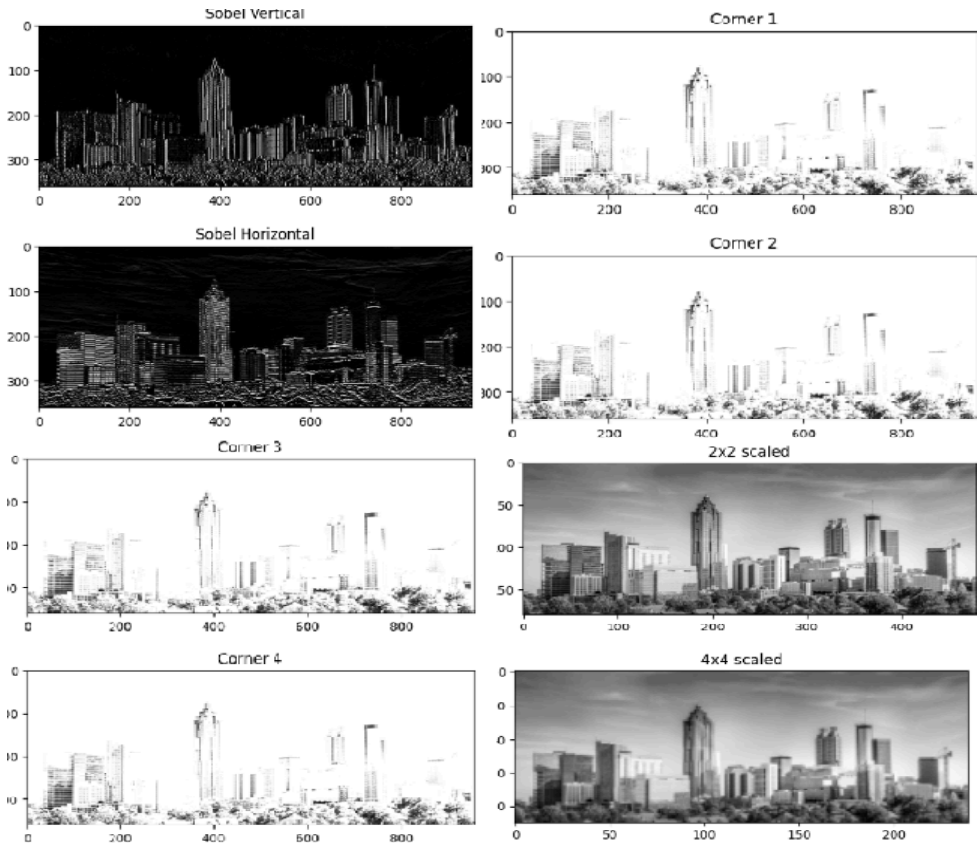


Figure 1 shows the output of several filters applied to the building image. The Sobel filters picked up edges in both directions, vertical edges are visible along the buildings, and horizontal ones show up more along the skyline. The corner detection filters each look for a specific corner shape, so the bright areas shift depending on which direction is being detected. The 2x2 and 4x4 scaled images show what happens after blurring and downsampling, the image gets smaller and loses detail, but the overall structure is still visible.

4.2 Part 2 Results

For Part 2, I built and trained a CNN model for CIFAR-10 image classification. The final model used three convolutional layers, max-pooling after each convolution block, a flatten layer, and two fully connected layers. During training, both training loss and validation loss generally decreased over time, which showed that the model was learning useful image features as training progressed. In the 10-epoch run, the training loss dropped from about 1.435 in the first epoch to about 0.463 by epoch 10, while the validation loss decreased from about 1.092 to about 0.729. The model also saved the best weights whenever validation loss improved.

After training, the model was tested on unseen data. The final test loss for the 10-epoch run was about 0.750, and the overall test accuracy was 75% on the CIFAR-10 test set. Some classes performed better than others. For example, ship, truck, automobile, and airplane had stronger results, while bird and cat were lower compared to the other classes.

Figure 2. Training output for the 10-epoch CNN run

```
Epoch: 1        Training Loss: 1.435148         Validation Loss: 1.091716
Validation loss decreased (inf --> 1.091716).  Saving model ...
Epoch: 2        Training Loss: 1.050239         Validation Loss: 0.948706
Validation loss decreased (1.091716 --> 0.948706).  Saving model ...
Epoch: 3        Training Loss: 0.878433         Validation Loss: 0.824983
Validation loss decreased (0.948706 --> 0.824983).  Saving model ...
Epoch: 4        Training Loss: 0.774595         Validation Loss: 0.765522
Validation loss decreased (0.824983 --> 0.765522).  Saving model ...
Epoch: 5        Training Loss: 0.700134         Validation Loss: 0.774150
Epoch: 6        Training Loss: 0.640122         Validation Loss: 0.750335
Validation loss decreased (0.765522 --> 0.750335).  Saving model ...
Epoch: 7        Training Loss: 0.585547         Validation Loss: 0.718083
Validation loss decreased (0.750335 --> 0.718083).  Saving model ...
Epoch: 8        Training Loss: 0.535674         Validation Loss: 0.711632
Validation loss decreased (0.718083 --> 0.711632).  Saving model ...
Epoch: 9        Training Loss: 0.495821         Validation Loss: 0.736971
Epoch: 10       Training Loss: 0.463320         Validation Loss: 0.728981
```

Figure 3. Testing results for the trained CNN model

```
Test Loss: 0.750363

Test Accuracy of airplane: 83% (836/1000)
Test Accuracy of automobile: 84% (848/1000)
Test Accuracy of  bird: 59% (594/1000)
Test Accuracy of   cat: 54% (548/1000)
Test Accuracy of  deer: 71% (716/1000)
Test Accuracy of   dog: 68% (683/1000)
Test Accuracy of  frog: 75% (752/1000)
Test Accuracy of horse: 81% (813/1000)
Test Accuracy of  ship: 88% (886/1000)
Test Accuracy of truck: 85% (857/1000)

Test Accuracy (Overall): 75% (7533/10000)
```

During training, both training loss and validation loss generally decreased over time, which showed that the model was learning useful image features as training progressed, as shown in **Figure 2**.

After training, the model was tested on unseen data, and the final test loss and overall test accuracy are shown in **Figure 3**.

4.3 Accuracy

The final CNN model achieved an overall test accuracy of 75%, which is a solid result for this assignment and is above the notebook's suggested "good" target of around 70% or more. Looking at the class-level results, the highest accuracies were ship at 88%, truck at 85%, automobile at 84%, and airplane at 83%. Lower-performing classes included cat at 54% and bird at 59%, which suggests that some classes were harder for the model to separate than others.

I also did a smaller comparison run using 5 epochs. That run reached 74% overall test accuracy, which was very close to the 10-epoch result. Because of that, increasing from 5 epochs to 10 epochs did not create a huge improvement in final accuracy in my case, even though the 10-epoch run still gave the slightly better final result.

4.4 Time Taken for Training

Training was done on CPU since CUDA was not available in my environment. The full 10-epoch run took approximately 14 minutes. I also ran a shorter 5-epoch comparison run which took approximately 6 minutes.

4.5 Number of Parameters

The CNN model had a total of 1,147,466 trainable parameters. Most of these parameters came from the first fully connected layer, which mapped the flattened 2048 features into 512 outputs. This shows that even though the convolution layers are important for feature extraction, a large part of the model complexity was in the fully connected section at the end.

This parameter count also helps explain why the model was capable of learning useful patterns from the CIFAR-10 images while still staying small enough to train for this assignment.

Figure 4. Training and Validation Loss curve for the 10-epoch CNN run

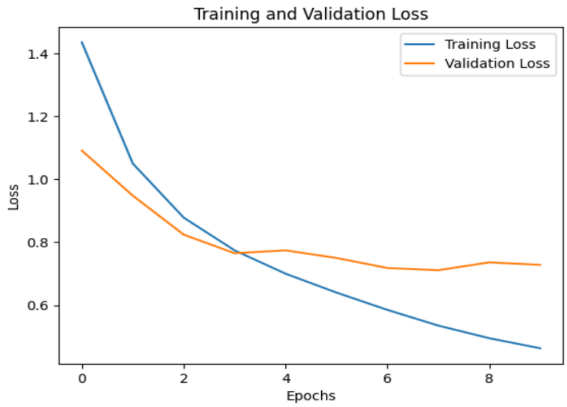


Figure 4 shows the training and validation loss over 10 epochs, where both curves decrease over time, indicating that the model was learning effectively throughout training.

Table 1. CNN architecture table

Layer	Output Size	Parameters
Input	32×32×3	0
Conv2d(3,32,3,p=1)	32×32×32	896
MaxPool2d(2,2)	16×16×32	0
Conv2d(32,64,3,p=1)	16×16×64	18,496
MaxPool2d(2,2)	8×8×64	0
Conv2d(64,128,3,p=1)	8×8×128	73,856
MaxPool2d(2,2)	4×4×128	0
Flatten	2048	0
Linear(2048,512)	512	1,049,088
Linear(512,10)	10	5,130
Total		**1,147,466**

The final CNN architecture is also presented in **Table 1**, which summarizes the input size, layer progression, and output dimensions.

The CNN architecture and parameter counts were also checked manually using the standard formulas for convolutional and fully connected layers.

Below formula was used to calculate **Conv2d** and **MaxPool**

$$out(N_i, C_j, h, w) = \max_{m=0,\ldots,kH-1} \max_{n=0,\ldots,kW-1} \mathrm{input}(N_i, C_j, \mathrm{stride}[0] \times h + m, \mathrm{stride}[1] \times w + n)$$

Here is the simplified version for both:

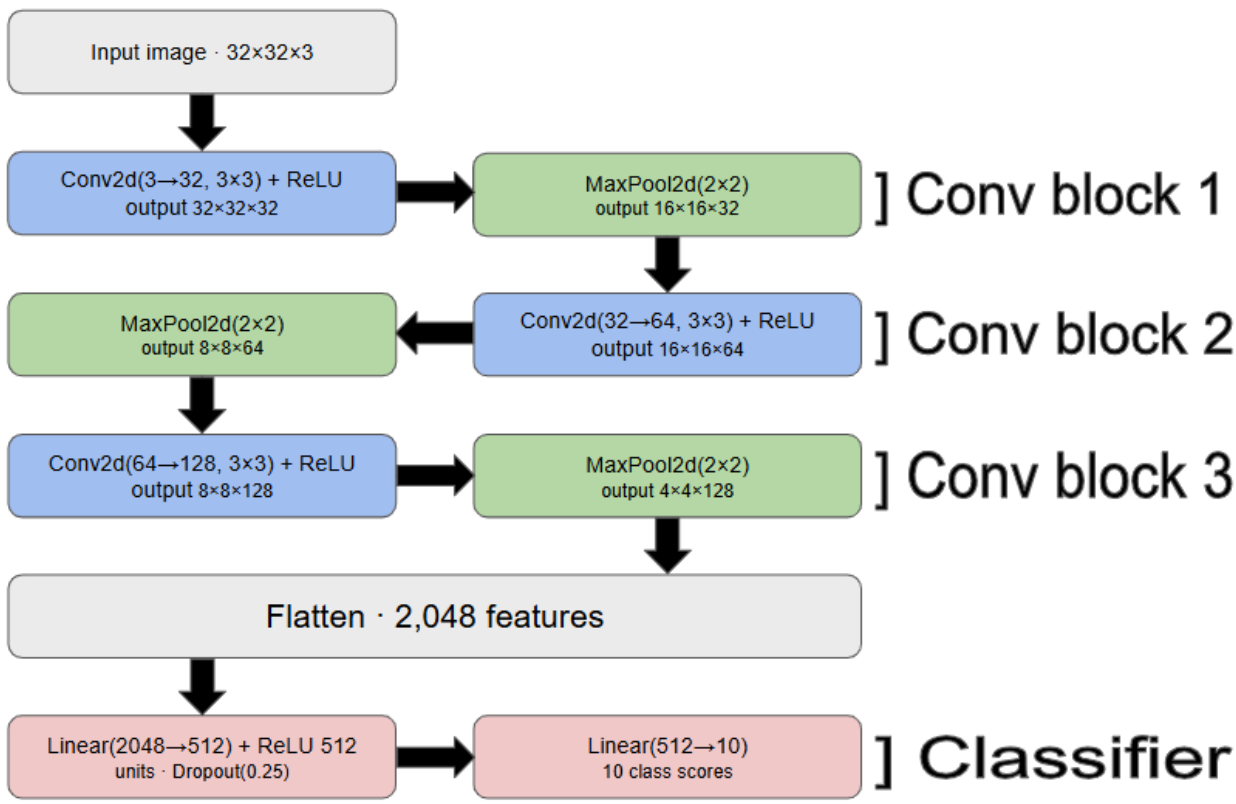
Conv2d:

$$Output = \left\lfloor \frac{Input - Kernel + 2 \times Padding}{Stride} + 1 \right\rfloor$$

MaxPool2d (since padding=0 and kernel=stride):

$$Output = \left\lfloor \frac{Input}{Stride} \right\rfloor$$

Graph 1. Graphical Illustration of CNN architecture



4.6 What Helped and What Did Not

A few things clearly helped the model perform better. First, using three convolutional layers gave the network enough depth to learn more useful image features. Second, using max-pooling after each convolution reduced the spatial size in a clean way and helped the network focus on stronger patterns. Third, the training and validation losses both moved downward overall, which showed that the training setup was working well. The final model also benefited from using CrossEntropyLoss and Adam with a learning rate of 0.001, which is what I used in the notebook.

One thing I found interesting was that increasing the training from 5 epochs to 10 epochs did not make a dramatic difference in overall accuracy. The 5-epoch run already reached 74%, and the 10-epoch run reached 75%, so the extra training only gave a small improvement in this case. This suggests that the model was already learning useful features fairly early, and that simply adding more epochs does not always lead to a big jump in performance.

Another thing that did not help equally across all classes was the general difficulty of the dataset itself. Some classes were easier for the model to recognize, while others were more confusing. This can be seen in the lower class accuracies for categories like cat and bird compared to ship or truck. Overall, the main takeaway was that architecture choices and a solid training setup helped more than just increasing epochs alone.

Figure 5. Training output for the 5-epoch comparison run

```
Epoch: 1        Training Loss: 1.413317         Validation Loss: 1.093534
Validation loss decreased (inf --> 1.093534).  Saving model ...
Epoch: 2        Training Loss: 1.013845         Validation Loss: 0.900002
Validation loss decreased (1.093534 --> 0.900002).  Saving model ...
Epoch: 3        Training Loss: 0.845542         Validation Loss: 0.818879
Validation loss decreased (0.900002 --> 0.818879).  Saving model ...
Epoch: 4        Training Loss: 0.747952         Validation Loss: 0.771148
Validation loss decreased (0.818879 --> 0.771148).  Saving model ...
Epoch: 5        Training Loss: 0.662754         Validation Loss: 0.721323
Validation loss decreased (0.771148 --> 0.721323).  Saving model ...
```

Figure 6. Test results for the 5-epoch comparison run

```
Test Loss: 0.741378

Test Accuracy of airplane: 71% (719/1000)
Test Accuracy of automobile: 85% (854/1000)
Test Accuracy of  bird: 54% (549/1000)
Test Accuracy of   cat: 55% (558/1000)
Test Accuracy of  deer: 76% (764/1000)
Test Accuracy of   dog: 62% (628/1000)
Test Accuracy of  frog: 84% (846/1000)
Test Accuracy of horse: 77% (773/1000)
Test Accuracy of  ship: 88% (881/1000)
Test Accuracy of truck: 85% (855/1000)

Test Accuracy (Overall): 74% (7427/10000)
```

I also ran a smaller comparison using 5 epochs. The results shown in Figure 5 and Figure 6 were very close to the 10-epoch version, which suggests that increasing the training from 5 to 10 epochs did not create a major improvement in this case.

5. Conclusion

5.1 Summary Of Findings

Overall, this assignment helped me understand both basic image filtering and CNN-based image classification. In Part 1, I completed the image filtering tasks and was able to see how different filters changed the appearance of the images. That part was useful for understanding how image features can be enhanced or modified directly.

In Part 2, I built and trained a CNN for image classification. The model trained successfully, and the final results showed that it was able to learn useful features from the dataset. The final 10-epoch run reached about 75% test accuracy, and the model had a total of 1,147,466 trainable parameters. I also compared a 5-epoch run with the 10-epoch run and found that the results were fairly close, which showed that increasing the epochs did not create a huge change in accuracy in this case.

5.2 Personal Insights

This assignment gave me a better understanding of how image processing and CNNs connect to each other. Part 1 helped me see the more direct side of working with images, while Part 2 showed how a neural network can automatically learn features during training. I also understood more clearly how training loss, validation loss, accuracy, and parameter count all help describe the model's performance.

One thing I found interesting was that improving a model is not always just about training longer. I first expected that going from 5 epochs to 10 epochs would make a much bigger difference, but in my results the improvement was small. That showed me that model performance depends on multiple factors, not only the number of epochs.

5.3 Future Consideration

If I continued working on this assignment, I would try a few more changes to see whether the model performance could improve further. For example, I would compare SGD and Adam more directly, test different learning rates, and possibly try small architecture changes such as adjusting the number of filters or fully connected units. I would also look at whether simple data augmentation could help improve performance on the harder classes.

Overall, this assignment was helpful because it combined visual image processing with neural network training, and it gave me a better understanding of both the basic and more advanced sides of image-based machine learning.

6. References

- Course-provided Jupyter notebook: image_filter.zip, CSCI 611 Applied Machine Learning, Spring 2026.
- Course-provided Jupyter notebook: build_cnn.zip, CSCI 611 Applied Machine Learning, Spring 2026.
- Machine Learning with Pytorch and Scikit-Learn: Develop Machine Learning and Deep Learning Models with Python
- CSCI 611 Course Lecture Notes.Pytorch.pdf (Canvas PDF).
- CSCI 611 Course Lecture Notes. CNN.pdf (Canvas PDF).